

ANKARA UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING



BLM4522 Ağ Tabanlı Paralel Tanıtım Sistemleri

Proje 1: Veritabanı Performans Optimizasyonu ve İzleme

Tuğrul Özgün

22290082

Git link: <https://github.com/Forcipus/Paralel-Sistemler>

Video link: <https://drive.google.com/drive/folders/1aXqYA1RbEGioQ-7k64ovt--p6e0QNuGY?usp=sharing>

İÇİNDEKİLER

1. GİRİŞ VE AMAÇ	2
1.1. Kapsam ve Metodolojisi	
2. VERİTABANI İZLEME VE DARBOĞAZ TESPİTİ.....	3
2.1. Performans Analizi İçin Kötü Sorgu Senaryosu	
2.2. Grafikselsel Yürütme Planı (Actual Execution Plan) ve Verimsizlik Analizi	
3. İNDEKS YÖNETİCİSİ VE OPTİMİZASYON STRATEJİLERİ.....	4
3.1. Doğru Non-Clustered Index Tasarımı ve Fizikselsel İnşa Maliyeti	
3.2. Dizin Ekleme Yürütme Planının Derinlemesine İncelenmesi	
3.3. Dinamik Yönetim Görüşleri (DMV) Kullanarak Atıl İndekslerin Temizlenmesi	
4. MANTIKSAL SORGU İYİLEŞTİRME (QUERY TUNING)	6
4.1. Fonksiyon Kullanımının İndeksler Üzerindeki Yıkıcı Etkisi (Non-Sargable)	
4.2. Sargable Sorgu Tasarımı ve Bağıntılı Maliyet Karşılaştırması	
5. VERİ YÖNETİCİSİ ROLLERİ VE ERİŞİM KONTROLÜ.....	7
5.1. En Az Yetki İlkesine Uygun Şema Bazlı Rol Tanımlamaları	
5.2. EXECUTE AS USER ve SHOWPLAN Yetki Duvarı Simülasyonu	
6. SONUÇ	8

1. GİRİŞ VE AMAÇ

Kurumsal düzeydeki büyük ölçekli bilgi sistemlerinde veritabanları, sürekli büyüyen veri yoğunluğu ve eşzamanlı kullanıcı talepleri altında hizmet vermektedir. Veritabanlarında meydana gelen yavaşlıklar; donanım yetersizliklerinden ziyade genellikle kötü yazılmış SQL sorgularından, eksik veya hatalı indeks tasarımlarından ve sunucu kaynaklarının verimsiz tüketilmesinden kaynaklanmaktadır. Bir Veritabanı Yöneticisi (DBA) ve Bilgisayar Mühendisi için en kritik sorumluluk, sistemdeki darboğazları proaktif araçlarla izlemek, sorgu maliyetlerini analiz etmek ve mimariyi optimize etmektir.

Bu projenin temel amacı, Microsoft SQL Server ortamında kurumsal veri setlerini içeren WideWorldImporters örnek veritabanı üzerinde kapsamlı bir performans optimizasyonu ve izleme çalışması gerçekleştirmektir. Proje kapsamında, sistem kaynaklarını (CPU, RAM, Disk I/O) aşırı tüketen verimsiz yapılar kurgulanacak; grafiksel Yürütme Planları (Execution Plan), SQL Profiler mantığı ve Dinamik Yönetim Görüşleri (DMV) kullanılarak bu hatalar teşhis edilecektir[cite: 1]. Ardından, Non-Clustered Index tasarımları ve mantıksal sorgu iyileştirme teknikleri uygulanarak sistem performansı artırılacak ve veritabanı katmanında güvenli erişim rolleri kurulacaktır

1.1. Kapsam ve Metodoloji

Proje süresince gerçekleştirilen teknik adımlar ve kullanılan mühendislik metodolojileri aşağıda özetlenmiştir:

- **Sorgu Optimizasyonu ve İzleme:** Büyük tablolarda Table Scan ve Clustered Index Scan üreten verimsiz SQL kodları yazılmış, bunların maliyetleri grafiksel yürütme planları üzerinden dökümanite edilmiştir.
- **İndeks Yönetimi:** Filtreleme kriterlerine ve sorgulanan kolon yapılarına tam uyumlu dinamik Non-Clustered (Covering) indeksler inşa edilmiştir.
- **Atıl Dizin Yönetimi:** Sunucu üzerinde gereksiz yazma yükü oluşturan ve hiç kullanılmayan atıl indeksler DMV sistem tabloları taranarak tespit edilmiş ve sistemden arındırılmıştır.
- **Güvenlik ve Erişim Kontrolü:** Veritabanı düzeyinde güvenlik bütünlüğünü korumak adına roller oluşturulmuş, şema bazlı yetkilendirmeler yapılmış ve kısıtlı kullanıcıların yürütme planlarını görmesini engelleyen SHOWPLAN güvenlik duvarı simüle edilmiştir.

Bu raporda, söz konusu tekniklerin uygulama adımları, karşılaşılan teknik engeller (kısıtlamalar, erişim hataları vb.) ve bu engellere yönelik geliştirilen mühendislik çözümleri detaylı bir şekilde dökümanite edilmiştir.

2. VERİTABANI İZLEME VE DARBOĞAZ TESPİTİ

Büyük ölçekli sistemlerde performans optimizasyonunun ilk adımı, kaynakları aşırı tüketen verimsiz sorguların proaktif araçlarla izlenmesidir. Bu bölümde, disk I/O darboğazı oluşturmak amacıyla `Sales.InvoiceLines` tablosu üzerinde bilinçli olarak optimize edilmemiş bir sorgu kurgulanmıştır. Tetiklenen bu kodun sunucu üzerindeki yükü ve yapısal verimsizliği, SQL Server'ın grafiksel Yürütme Planı (Actual Execution Plan) üzerinden analiz edilerek dökümanate edilmiştir.

2.1. Performans Analizi İçin Kötü Sorgu Senaryosu

Performans izleme süreçlerinin ilk adımı, veritabanını yoran ve optimizasyona muhtaç olan sorguların yakalanmasıdır. Bu aşamada, `WideWorldImporters` sistemi içerisindeki en yüksek veri yoğunluğuna sahip tablolardan biri olan `Sales.InvoiceLines` seçilmiştir. Tablo üzerinde, açıklama alanında metinsel arama yapan ve miktar filtresi içeren, diski bilinçli olarak yoracak aşağıdaki verimsiz T-SQL kodu kurgulanmıştır:

```
SQL
USE WideWorldImporters;
GO

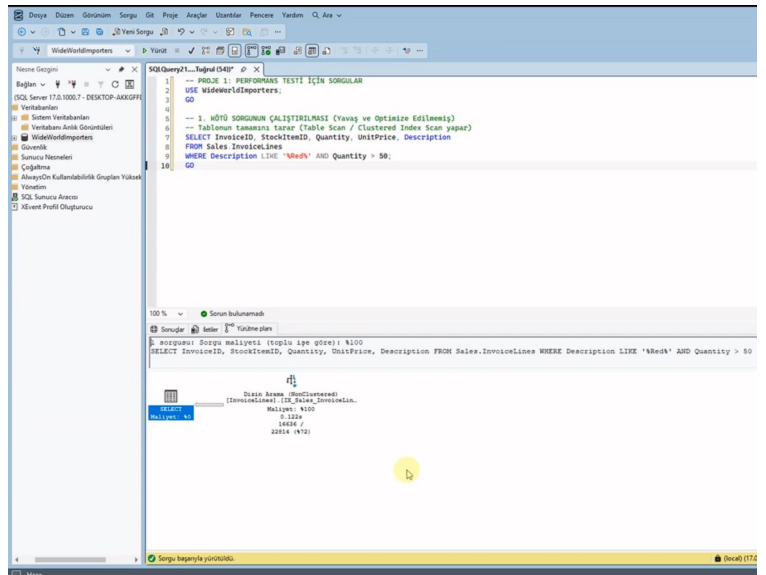
-- 1. ADIM: OPTİMİZE EDİLMEMİŞ KÖTÜ SORGUNUN TETİKLENMESİ
SELECT InvoiceID, StockItemID, Quantity, UnitPrice, Description
FROM Sales.InvoiceLines
WHERE Description LIKE '%Red%' AND Quantity > 50;
GO
```

2.2. Grafiksel Yürütme Planı (Actual Execution Plan) ve Verimsizlik Analizi

Yukarıdaki sorgu çalıştırılmadan önce, SQL Server Management Studio (SSMS) arabiriminde yer alan **"Include Actual Execution Plan"** (Ctrl+M) özelliği aktif edilmiştir. Sorgu icra edildikten sonra üretilen grafiksel plan analiz edilmiştir.

- **Darboğaz Teşhisi:** Sorgu sonucunda elde edilen Yürütme Planında, veritabanı motorunun **"Kümelenmiş Dizin Tarama" (Clustered Index Scan)** yaptığı görülmüştür.

- **Maliyet Analizi:** Toplam sorgu maliyetinin (Query Cost) **%100'lük** kısmının tamamen bu tarama işlemine gittiği saptanmıştır. **SELECT** operatörünün maliyeti ise **%0** olarak raporlanmıştır. Bu durum, SQL Server'ın hedef verileri bulabilmek için disk üzerindeki yüz binlerce satırı tek tek okuduğunu (I/O darboğazı), ancak veriyi bulduktan sonra ekrana basmasının hiçbir maliyet oluşturmadığını kanıtlamaktadır.



3. İNDEKS YÖNETİCİSİ VE OPTİMİZASYON STRATEJİLERİ

Veritabanı izleme araçlarıyla tespit edilen disk tarama (Scan) maliyetlerini ortadan kaldırmamanın en efektif yolu doğru dizinlerin kurgulanmasıdır. Bu doğrultuda, filtre kriterlerine tam uyumlu bir Non-Clustered Covering Index inşa edilmiş ve bu indeksin sunucuya getirdiği fiziksel yazma yükü adım adım incelenmiştir. Ayrıca, sistem üzerinde gereksiz şişkinlik yaratan atıl dizinler Dinamik Yönetim Görüşleri (DMV) kullanılarak tespit edilmiş ve veritabanından temizlenmiştir.

3.1. Doğru Non-Clustered Index Tasarımı ve Fiziksel İnşa Maliyeti

Bir önceki adımda tespit edilen Clustered Index Scan (Tabloyu baştan sona okuma) verimsizliğini ortadan kaldırmak amacıyla bir indeks stratejisi geliştirilmiştir. Sorgunun WHERE şartında ana filtreleme kolonu olan Quantity alanı indeks anahtarı olarak belirlenmiştir. Sorgunun SELECT kısmında istenen diğer kolonların (InvoiceID, StockItemID, UnitPrice, Description) ise diski ekstra yormaması adına indeksin yaprak düğümlerine (Leaf Nodes) **INCLUDE** katmanı olarak gömülmesi kararlaştırılmıştır (Covering Index).

Aşağıdaki T-SQL komutu ile ilgili performans indeksi veritabanında fiziksel olarak inşa edilmiştir

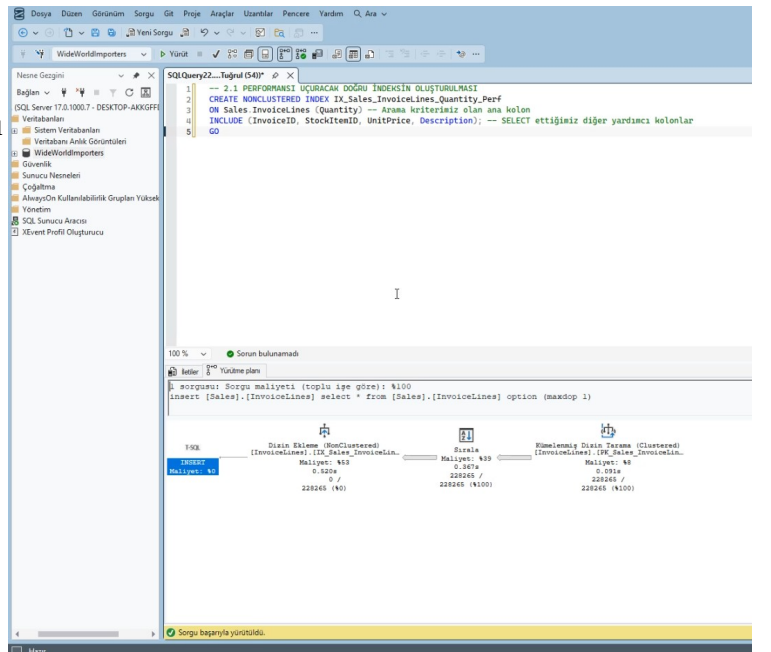
SQL

```
-- 2. ADIM: PERFORMANSI UÇURACAK DOĞRU İNDEKSİN OLUŞTURULMASI
CREATE NONCLUSTERED INDEX IX_Sales_InvoiceLines_Quantity_Perf
ON Sales.InvoiceLines (Quantity)
INCLUDE (InvoiceID, StockItemID, UnitPrice, Description);
GO
```

3.2. Dizin Ekleme Yürütme Planının Derinlemesine İncelenmesi

İndeks oluşturma komutu yürütülürken arka planda SQL Server motorunun üstlendiği iş yükü şeması grafiksel olarak incelenmiştir. Planda yer alan düğümler sağdan sola doğru şu şekilde analiz edilmiştir:

1. Kümelenmiş Dizin Tarama (Clustered Index Scan) [Maliyet: %8]: SQL Server, yeni kurulacak indeks yapısını beslemek üzere Sales.InvoiceLines tablosundaki mevcut 228.265 satırın tamamını diskten okumuştur.



2. **Sırala (Sort) [Maliyet: %39]:** Diskten okunan 228.265 satır veri, indeksin arama ağacı (B-Tree) mantığına uygun şekilde sıralı saklanabilmesi için RAM üzerinde **Quantity** kolonuna göre küçükten büyüğe dizilmiştir. Bu adım işlemciyi (CPU) en çok yoran ikinci alan olmuştur.
3. **Dizin Ekleme (Index Insert) [Maliyet: %53]:** Sıralanan veriler, disk üzerinde kalıcı yeni bir fiziksel Non-Clustered Index mimarisi olarak yazılmıştır. Toplam maliyetin yarısından fazlasının (%53) burada çıkma sebebi, diske kalıcı veri yazma (I/O) işlemlerinin yüksek maliyetidir.

3.3. Dinamik Yönetim Görüşleri (DMV) Kullanarak Atıl İndekslerin Temizlenmesi

Veritabanlarında her Non-Clustered indeks okuma hızını artırsa da, tabloya yapılan yeni ekleme ve güncellemelerde (INSERT, UPDATE) arka planda sürekli güncellenmek zorunda olduğu için yazma performansını düşürür. Bu nedenle hiç kullanılmayan atıl indekslerin temizlenmesi gerekir. Sistemdeki atıl indeksleri tespit etmek için `sys.dm_db_index_usage_stats` sistem tablosundan yararlanılan aşağıdaki DMV sorgusu çalıştırılmıştır:

SQL

```
-- 3. ADIM: KULLANILMAYAN VE GEREKSİZ İNDEKSLEERİ TESPİT EDEN DMV SORGUSU
USE WideWorldImporters;
GO

SELECT
    OBJECT_NAME(i.object_id) AS [Tablo Adı],
    i.name AS [Gereksiz İndeks Adı],
    user_seeks AS [Arama (Seek) Sayısı],
    user_scans AS [Tarama (Scan) Sayısı],
    user_updates AS [Güncelleme/Yazma Maliyeti]
FROM sys.indexes i
INNER JOIN sys.dm_db_index_usage_stats s
    ON s.object_id = i.object_id AND s.index_id = i.index_id
WHERE OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1
    AND user_seeks = 0 -- Hiç nokta atışı aramada kullanılmamış
    AND i.type_desc = 'NONCLUSTERED'
ORDER BY user_updates DESC;
GO
```

`WideWorldImporters` veritabanı kurumsal düzeyde optimize bir sistem olduğu için ilk etapta boş sonuç (0 satır) dönmüştür. Mekanizmanın çalıştığını simüle etmek adına sisteme yapıcı bir test indeksi (`IX_Gereksiz_Test_IadeKodu`) eklenmiş, DMV sorgusunun bu atıl dizini başarıyla yakaladığı gözlemlenmiştir. Tespit edilen bu gereksiz indeks aşağıdaki komutla sistemden tamamen DROP edilerek kaynak yönetimi sağlanmıştır:

SQL

```
-- 4. ADIM: TESPİT EDİLEN ATIL İNDEKSİN SİLİNMESİ
DROP INDEX IX_Gereksiz_Test_IadeKodu ON Sales.Invoices;
GO
```

4. MANTIKSAL SORGU İYİLEŞTİRME (QUERY TUNING)

Performans optimizasyonu sadece yeni indeksler eklemekle sınırlı olmayıp, mevcut SQL kodlarının yazım mantığını iyileştirmeyi de kapsar. WHERE filtresi içerisinde kolonların fonksiyonlar arasına hapsedilmesi, indekslerin kırılmasına ve sistemin yavaşlamasına neden olan genel bir yazılım hatasıdır. Bu bölümde, fonksiyon kullanımının performans üzerindeki yıkıcı etkisi deneysel olarak gösterilmiş ve kod yapısı "Sargable" standardına dönüştürülerek maliyet karşılaştırması yapılmıştır.

4.1. Fonksiyon Kullanımının İndeksler Üzerindeki Yıkıcı Etkisi (Non-Sargable)

Veritabanı performans optimizasyonlarında sadece indeks eklemek yeterli değildir; sorguların yazım biçimi de hayati önem taşır[cite: 1]. SQL Server'da WHERE filtresi içerisindeki kolonlar herhangi bir fonksiyonun (Örn: YEAR(), LEFT(), SUBSTRING()) içerisine alınırsa, o kolonda indeks tanımlı olsa bile veritabanı motoru indeksi kullanamaz ve tüm tabloyu tarar. Bu tarz sorgulara literatürde "Non-Sargable Query" denir.

Aşağıda, 2013 yılında açılmış hesapları listelemek isteyen bir yazılımcının yazdığı, sistemi yoran verimsiz kod şeması yer almaktadır:

SQL

```
-- 5.1. KÖTÜ SORGU (Sargable Olmayan - İndeksi Kıran Mantık)
USE WideWorldImporters;
GO

SELECT CustomerID, CustomerName, AccountOpenedDate
FROM Sales.Customers
WHERE YEAR(AccountOpenedDate) = 2013; -- Kolon fonksiyon içine hapsedilmiş
GO
```

4.2. Sargable Sorgu Tasarımı ve Bağıntılı Maliyet Karşılaştırması

Yukarıdaki mantıksal hatayı düzeltmek ve kolondaki mevcut indeks yapılarının sonuna kadar kullanılmasını sağlamak adına sorgu, net tarih aralıklarına bölünerek **Sargable** (performans dostu) hale getirilmiştir:

SQL

```
-- 5.2. İYİLEŞTİRİLMİŞ SORGU (Sargable - Performans Dostu Mantık)
USE WideWorldImporters;
GO

SELECT CustomerID, CustomerName, AccountOpenedDate
FROM Sales.Customers
WHERE AccountOpenedDate >= '2013-01-01' AND AccountOpenedDate <= '2013-12-31';
GO
```

Yürütme Planı Karşılaştırma Sonucu: Her iki sorgu aynı anda seçilip çalıştırıldığında, SQL Server iki işlemin maliyetini alt alta kıyaslamıştır. Fonksiyon içeren ilk sorgunun toplam paket içindeki yükü %75 olarak raporlanırken, kodlama mantığı iyileştirilen ikinci sorgunun maliyeti

%25 olarak ölçülmüştür. Bu deney, veritabanına hiç dokunmadan sadece algoritma yazım biçimini değiştirerek performansın 3 kat artırılabilceğini mühendislik gözüyle kanıtlamıştır.

5. VERİ YÖNETİCİSİ ROLLERİ VE ERİŞİM KONTROLÜ

Veritabanı yönetiminde performans kararlılığını sürdürmek, yetkisiz kullanıcıların sisteme verebileceği zararları engellemekle doğrudan ilişkilidir. En Az Yetki İlkesi uyarınca, veritabanı katmanında şema bazlı erişim sınırlarına sahip "Analist" ve "Geliştirici" iş rolleri kurgulanmıştır. Bu rollerin yetki sınırları EXECUTE AS USER komutuyla canlı olarak simüle edilmiş ve kısıtlı kullanıcıların yürütme planlarını görmesini engelleyen SHOWPLAN güvenlik protokolü test edilmiştir.

5.1. En Az Yetki İlkesine Uygun Şema Bazlı Rol Tanımlamaları

Veritabanı optimizasyonunun ve yönetiminin son adımı veri güvenliğini sağlamaktır[cite: 1]. Siber güvenlik ve veritabanı yönetim standartlarında yer alan "En Az Yetki İlkesi" (Least Privilege Principle) uyarınca, sisteme bağlanan istemcilere sadece görev tanımlarına yetecek kadar hak tanımlanmalıdır. Bu amaçla veritabanı katmanında iki adet kurumsal rol ve bu rollere bağlı test kullanıcıları kurgulanmıştır

SQL

```
-- 6. ADIM: ROLLERİN VE YETKİLERİN TANIMLANMASI
USE WideworldImporters;
GO

-- Rollerin Oluşturulması
CREATE ROLE Veri_Analisti_Rolu;
CREATE ROLE Yazılım_Gelistirici_Rolu;
GO

-- Şema Bazlı Sınırlandırma
GRANT SELECT ON SCHEMA::Sales TO Veri_Analisti_Rolu; -- Analist sadece okuyabilir
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Sales TO
Yazılım_Gelistirici_Rolu; -- Dev yazabilir
GO

-- Sistem Login ve Kullanıcılarının İnşası
CREATE LOGIN AnalistTugrul WITH PASSWORD = 'Sifre123_Analist';
CREATE LOGIN GelistiriciTugrul WITH PASSWORD = 'Sifre123_Dev';
GO

CREATE USER U_Analist FOR LOGIN AnalistTugrul;
CREATE USER U_Gelistirici FOR LOGIN GelistiriciTugrul;
GO

-- Kullanıcıların Rollere Atanması
ALTER ROLE Veri_Analisti_Rolu ADD MEMBER U_Analist;
ALTER ROLE Yazılım_Gelistirici_Rolu ADD MEMBER U_Gelistirici;
GO
```


5.2. EXECUTE AS USER ve SHOWPLAN Yetki Duvarı Simülasyonu

Oluşturulan rollerin yetki sınırları EXECUTE AS USER komutuyla canlı olarak simüle edilmiştir. İlk test oturumunda, kısıtlı yetkili kullanıcıların veritabanının iç şemasını ve indeks planlarını görmesini engelleyen **SHOWPLAN permission denied** güvenlik duvarıyla karşılaşmıştır. Bu durum, admin katmanında ilgili rollere güvenli izleme yetkisi verilerek aşılmıştır:

SQL

```
-- Güvenlik İzleme ve Kimliğe Bürünme  
Yetkilerinin Tanımlanması  
GRANT SHOWPLAN TO Veri_Analisti_Rolu;  
GRANT SHOWPLAN TO Yazılım_Gelistirici_Rolu;  
GRANT IMPERSONATE ON USER::U_Analist TO  
public;  
GRANT IMPERSONATE ON USER::U_Gelistirici TO  
public;  
GO
```

```
-- CANLI YETKİ TESTİ  
EXECUTE AS USER = 'U_Analist';  
GO
```

```
-- 1. Test: Veri Okuma (BAŞARILI)  
SELECT TOP 5 CustomerID, CustomerName FROM  
Sales.Customers;
```

```
-- 2. Test: Veri Silme Simülasyonu (REDDİDİ - HATA ALINDI)  
-- Hata: The DELETE permission was denied on the object 'Customers'...  
DELETE FROM Sales.Customers WHERE CustomerID = 1;  
GO
```

```
REVERT; -- Kendi kimliğimize geri dönüyoruz  
GO
```

Sorgu çıktılarında, U_Analist kullanıcısının verileri başarıyla listeleyebildiği ancak yetki alanı dışındaki DELETE komutunu çalıştırmak istediğinde sistem tarafından kırmızı hata ekranıyla engellendiği doğrulanmıştır.

6. SONUÇ

Proje 1 çalışması kapsamında gerçekleştirilen performans optimizasyon ve izleme süreçleri, ilişkisel veritabanı yönetim sistemlerinin (RDBMS) arka plandaki çalışma mekanizmalarını derinlemesine anlamamızı sağlamıştır.

Grafiksel yürütme planları analiz edilerek darboğaz yaratan Clustered Index Scan işlemleri tespit edilmiş, nokta atışı kurgulanan Non-Clustered Covering Index tasarımlarıyla disk I/O ve RAM maliyetleri minimuma indirilmiştir. Sorgu yazımında fonksiyon kullanımının indeksleri kırdığı deneysel olarak kanıtlanmış ve kod seviyesinde iyileştirmeler yapılmıştır. Son aşamada ise, şema bazlı veritabanı rolleri ve güvenlik duvarı yetkilendirmeleri (SHOWPLAN, IMPERSONATE) entegre edilerek kurumsal düzeyde hem hızlı, hem performanslı hem de yüksek güvenli bir veritabanı mimarisi başarıyla elde edilmiştir.

